Introduction

Information Retrieval (IR) is a field of study dedicated to the organization and retrieval of information from vast and diverse collections of data. In the context of digital information, IR involves the development of techniques and systems that enable users to efficiently access relevant information. Key tasks in IR include indexing, searching, and ranking documents based on their relevance to user queries.

This assignment delves into various aspects of Information Retrieval, exploring fundamental concepts and practical applications. Throughout the assignment, we will examine techniques for document indexing, retrieval models, and the evaluation of IR systems using metrics such as precision, recall, and mean average precision. By delving into these topics, we aim to gain a comprehensive understanding of the core principles that govern effective information retrieval.

In this lab, you are going to implement a standard document processing pipeline and then build a simple search engine based on it:

- Building various retrieval models.

In [1]:
```
import os
from collections import Counter
from tqdm import tqdm
import math
```

Indexing

An index plays a crucial role in facilitating efficient search and retrieval operations. It involves creating a structured data structure that maps terms to their corresponding documents, capturing the relationships between words and their occurrences within the collection.

By building the document index, we aim to enhance the effectiveness of our Information Retrieval system, enabling faster and more accurate retrieval of relevant documents. This process lays the foundation for various retrieval models and evaluation metrics that we will explore in subsequent steps.

The primary logic for indexing documents is implemented in this module using the following data structures:

- `doc_id_to_filename` : This dictionary establishes a mapping from document IDs to their corresponding file names within the index structure, aiding in the efficient retrieval and organization of documents.
- `filename_to_doc_id` : This dictionary facilitates the reverse mapping by associating file names with their respective document IDs, providing a quick lookup mechanism for identifying documents.
- `inverted_index` : This dictionary captures the inverted index, where each term is associated with a nested dictionary containing document IDs and their corresponding term frequencies. For instance: {term : {doc_id_1: term_freq_1, doc_id_2:term_freq_2, ...}}.
- `document_term_vector` : The document term vector delineates the terms and their frequencies within each document, structured as {doc_id : {term_1: term_freq_1, term_2:term_freq_2, ...}}, offering a detailed representation of the document's content.
- `collection_freq` : The collection frequency, formatted as {term_1: collection_freq_1, term_2:collection_freq_2, ...}, encapsulates the total occurrences of each term across all documents, considering multiple occurrences within a given document.
- `doc_freq` : This dictionary signifies the document frequency, specifying the number of documents in which each term appears, structured as {term_1: doc_freq_1, term_2:doc_freq_2, ...}.
- `doc_lengths` : This dictionary maps document IDs to their respective document lengths, represented as {doc_id : doc_length}, providing insights into the length of each document in the corpus and aiding in various information retrieval tasks.

In [2]:
```
class Indexer:
    """
    The Indexer class is responsible for managing document indexing.

    Attributes:
    - collection_path (str): The path to the document collection.
    - doc_id_to_filename (dict): Dictionary mapping document IDs to filenames.
    - filename_to_doc_id (dict): Dictionary mapping filenames to document IDs.
    - inverted_index (dict): Inverted index containing terms and their corresponding postings lists.
    - document_term_vector (dict): Vector representation of document terms.
    - collection_freq (dict): Dictionary storing the frequency of terms in the entire collection.
    - doc_freq (dict): Dictionary storing the document frequency of terms.
    - doc_lengths (dict): Dictionary storing the length of each document.
    - prep (Preprocessor): Preprocessor instance for text processing.

    Methods:
    - __init__(self, collection_path): Initializes the Indexer with the path to the document collection.
    """

    def __init__(self, collection_path):
        # Initialize attributes
        self.collection_path = collection_path
        self.doc_id_to_filename = {}
        self.filename_to_doc_id = {}
        self.inverted_index = {}
        self.document_term_vector = {}
        self.collection_freq = {}
```

```
        self.doc_freq = {}
        self.doc_lengths = {}
```

Testing index

Run the below code to create an index that you will use for testing your scoring functions.

In [3]:
```
indexer_test = Indexer(None)

indexer_test.inverted_index = {"do": {0: 2, 1: 3, 3: 3},
                               "be": {0: 2, 1: 2, 2: 2, 3: 2},
                               "or": {1: 1, 2: 1},
                               "to": {0: 4, 2: 2},
                               "da": {3: 3},
                               "am": {1: 1, 2: 2},
                               "not": {2: 1},
                               "i": {0: 2, 1: 3, 2: 2, 3: 2},
                               "it": {3: 2},
                               "therefore": {1: 1},
                               "is": {0: 2},
                               "let": {3: 2},
                               "think": {1: 1},
                               "what": {2: 1},
                               }
indexer_test.document_term_vector = {0:{'to': 4, 'do': 2, 'is': 2, 'be': 2},
                                     1:{'i': 2, 'think': 1, 'therefore': 1, 'am': 1, 'do': 3, 'be': 2},
                                     2:{'to': 2, 'be': 2, 'or': 1, 'not': 1, 'i': 2, 'am': 2, 'what': 1},
                                     3:{'do': 3, 'da': 3, 'let': 2, 'it': 2, 'be': 2}
                                    }
indexer_test.collection_freq = {"do": 8, "be": 8, "or": 2, "to": 6, "da": 3, "am": 3, "not": 1,
                                "i": 9, "it": 2, "therefore": 1, "is": 2, "let": 2, "think": 1, "what": 1
                               }
indexer_test.doc_freq = {"do": 3, "be": 4, "or": 2, "to": 2, "da": 1, "am": 2, "not": 1, "i": 4,
                         "it": 1, "therefore": 1, "is": 1, "let": 1, "think": 1, "what": 1
                        }
indexer_test.doc_lengths = {0: 10, 1: 11, 2: 10, 3: 12}

# prep_test = Preprocessor(stop_words={}, stemmer=None)
```

Scoring IR models

Scoring Information Retrieval (IR) models play a pivotal role in ranking and retrieving relevant documents from a vast collection based on a user's query. These models assign scores to documents, indicating their likelihood of being pertinent to the query. The primary objective is to differentiate between documents and present them in descending order of relevance, allowing users to efficiently access the most pertinent information. Scoring IR models employ various techniques, such as term frequency-inverse document frequency (TF-IDF) and Inverse Document Frequency to quantify the relevance of documents. These scores serve as a basis for ranking, ensuring that the most relevant documents are presented prominently in search results.

In this section, you will be tasked with implementing a range of Information Retrieval (IR) ranking functions, specifically focusing on Vector Space Ranking functions and Probabilistic Ranking functions as covered in both Week 1 and Week 2 of the lecture. Vector Space Models, such as TF-IDF and cosine similarity, facilitate the representation of documents and queries in a high-dimensional space, allowing for effective ranking based on similarity metrics. Additionally, Probabilistic Ranking functions, exemplified by models like BM25, consider the probabilistic distribution of terms within documents and queries to assign relevance scores. By engaging in the practical implementation of these IR ranking functions, you will gain hands-on experience in applying theoretical concepts to real-world scenarios, enhancing your understanding of information retrieval methodologies.

Vector Space Models

The Vector Space Model (VSM) is a fundamental framework in Information Retrieval (IR) that represents documents and queries as vectors in a high-dimensional space. In this model, each term in the corpus is assigned a unique dimension, and the presence or absence of terms is encoded as numerical values in the vectors.

Log frequency ranking

Log Frequency Ranking is a scoring mechanism commonly used in Information Retrieval to assess the relevance of documents to a given query. In this approach, the score for a document-query pair is determined by summing the logarithmically transformed frequencies of terms present in both the query and the document. Specifically, for each term shared between the query (q) and the document (d), the logarithm of the term frequency is calculated and added to the overall score. This logarithmic transformation helps mitigate the impact of overly frequent terms, providing a more nuanced and context-aware evaluation of relevance.

TASK 1

Below, you will need to implement Log frequency ranking function using the following mathematical expression:

$$score(q,d) = \sum_{t \in q \cap d} [1 + log_{10}(tf_{t,d})]$$

In [4]:
```
def vsm_log_freq(indexer, query_tokens, docs):
    """
    Calculate the Vector Space Model (VSM) scores using Log Frequency Ranking for a query and a given set of documents.

    Args:
        indexer (Indexer): An instance of the Indexer class containing the inverted index.
        query_tokens (dict): A dictionary representing the query with terms as keys and their frequencies as values.
        docs (list): A list of document IDs for which the scores need to be calculated.

    Returns:
        dict: A dictionary containing document IDs as keys and their corresponding scores as values.
    """
    results = {}

    ## START YOU CODE HERE
    for doc_id in docs:
        score = 0
        doc_terms = indexer.document_term_vector.get(doc_id, {})

        for term in query_tokens:
            if term in doc_terms:
                tf = doc_terms[term]
                score += 1 + math.log10(tf)

        if score > 0:
            results[doc_id] = score
    ## END

    return results
```

Let's now test the Log Frequency Ranking function by running the following cell:

In [5]:
```
assert vsm_log_freq(indexer_test,{'to':1, 'do':1}, [0,1,2,3]) == {0: 2.9030899869919438, 1: 1.4771212547196624, 2: 1.3010299
print("Success!")
assert vsm_log_freq(indexer_test,{'to':1, 'do':1, 'think':1}, [0,1,2,3]) == {0: 2.9030899869919438, 1: 2.4771212547196626, 2
print("Success!")
```

```
Success!
Success!
```

Jaccard ranking

Jaccard Ranking is a similarity measure commonly employed in Information Retrieval to assess the similarity between sets, particularly in the context of document retrieval. The Jaccard similarity coefficient is calculated as the size of the intersection of two sets (i.e., a query and a document) divided by the size of their union. Jaccard Ranking provides a simple yet effective way to quantify the overlap or commonality between two sets of terms, enabling the comparison of documents or queries based on their shared content.

TASK 2

Below, you will need to implement Jaccard ranking function using the following mathematical expression:

$$score(q,d) = \frac{|q \cap d|}{|q \cup d|}$$

In [6]:
```
def vsm_jaccard(indexer, query_tokens, docs):
    """
    Calculate the Jaccard similarity coefficients for a given set of documents using the Vector Space Model (VSM).

    Args:
        indexer (Indexer): An instance of the Indexer class containing the inverted index.
        query_tokens (dict): A dictionary representing the query with terms as keys and their frequencies as values.
        docs (list): A list of document IDs for which the scores need to be calculated.

    Returns:
        dict: A dictionary containing document IDs as keys and their corresponding scores as values.
    """
    results = {}

    ## START YOU CODE HERE
    query_terms = set(query_tokens.keys())

    for doc_id in docs:
        doc_terms = set(indexer.document_term_vector.get(doc_id, {}).keys())

        intersection = query_terms.intersection(doc_terms)
        union = query_terms.union(doc_terms)

        if len(union) > 0:
            score = len(intersection) / len(union)
```

```
            if score > 0:
                results[doc_id] = score
    ## END

    return results
```

Let's now test the Jaccard Ranking function by running the following cell:

In [7]:
```
assert vsm_jaccard(indexer_test,{'to':1, 'do':1}, [0,1,2,3]) == {0: 0.5, 1: 0.14285714285714285, 2: 0.125, 3: 0.166666666666
print("Success!")
assert vsm_jaccard(indexer_test,{'to':1, 'do':1, 'think':1}, [0,1,2,3]) == {0: 0.4, 1: 0.2857142857142857, 2: 0.111111111111
print("Success!")
```

```
Success!
Success!
```

Cosine ranking

The Cosine Similarity measure within the Vector Space Model is a metric used to assess the similarity between two vectors in a high-dimensional space, particularly in the context of Information Retrieval. Representing documents and queries as vectors, each dimension corresponds to a unique term, and the vector values denote the term's weight in the document or query. Cosine Similarity calculates the cosine of the angle between these vectors, providing a measure of similarity ranging from -1 (completely dissimilar) to 1 (identical). A score of 0 indicates orthogonality, implying no similarity. The method is advantageous as it is robust to document length variations and insensitive to the vector's magnitude. In Information Retrieval, higher Cosine Similarity scores indicate greater relevance, aiding in the ranking and retrieval of documents based on their similarity to user queries.

TASK 3

Following the SMART Notation (discussed in Week 1 slide 108) that denotes the combination in use in an engine, implement the cosine similarity function **using ltc.ltc (with log base 10)** using the following mathematical expression:

$$\text{Cosine}(\mathbf{d}, \mathbf{q}) = \frac{\sum_{i=1}^{n} d_i \cdot q_i}{\sqrt{\sum_{i=1}^{n} d_i^2} \cdot \sqrt{\sum_{i=1}^{n} q_i^2}}$$

In [8]:
```
def vsm_cosine(indexer, query_tokens, docs):
    """
    Calculate Vector Space Model (VSM) scores using the Cosine Similarity measure for a given set of documents.

    Args:
        indexer (Indexer): An instance of the Indexer class containing the inverted index.
        query_tokens (dict): A dictionary representing the query with terms as keys and their frequencies as values.
        docs (list): A list of document IDs for which the scores need to be calculated.

    Returns:
        dict: A dictionary containing document IDs as keys and their corresponding scores as values.
    """
    results = {}

    ## START YOU CODE HERE
    N = len(indexer.document_term_vector)

    # Building weighted query vector using ltc
    query_weights = {}
    for term, tf in query_tokens.items():
        if tf <= 0:
            continue
        if term not in indexer.doc_freq:
            continue

        df = indexer.doc_freq[term]
        if df == 0:
            continue

        tf_weight = 1 + math.log10(tf)
        idf_weight = math.log10(N / df)
        query_weights[term] = tf_weight * idf_weight

    # Query norm
    query_norm = math.sqrt(sum(w * w for w in query_weights.values()))

    # If query has no valid terms, all scores are 0
    if query_norm == 0:
        for doc_id in docs:
            results[doc_id] = 0.0
        return results

    # Normalizing query vector
    for term in query_weights:
        query_weights[term] /= query_norm

    # Scoring each document
```

```
    for doc_id in docs:
        if doc_id not in indexer.document_term_vector:
            results[doc_id] = 0.0
            continue

        doc_terms = indexer.document_term_vector[doc_id]

        # Building full weighted document vector using ltc
        doc_weights = {}
        for term, tf in doc_terms.items():
            if tf <= 0:
                continue
            if term not in indexer.doc_freq:
                continue

            df = indexer.doc_freq[term]
            if df == 0:
                continue

            tf_weight = 1 + math.log10(tf)
            idf_weight = math.log10(N / df)
            doc_weights[term] = tf_weight * idf_weight

        # Document norm
        doc_norm = math.sqrt(sum(w * w for w in doc_weights.values()))

        if doc_norm == 0:
            results[doc_id] = 0.0
            continue

        # Normalizing document vector
        for term in doc_weights:
            doc_weights[term] /= doc_norm

        # Cosine similarity = dot product of normalized vectors
        score = 0.0
        for term, q_weight in query_weights.items():
            score += q_weight * doc_weights.get(term, 0.0)

        results[doc_id] = score
    ## END

    return results
```

Let's now test the Cosine Ranking function by running the following cell:

In [9]:
```
assert vsm_cosine(indexer_test,{'to':1, 'do':1}, [0,1,2,3]) == {0: 0.5435531276342821, 1: 0.07674929484102105, 2: 0.34144579
print("Success!")
assert vsm_cosine(indexer_test,{'to':1, 'do':1, 'think':1}, [0,1,2,3]) == {0: 0.25876959324508964, 1: 0.6109385695061128, 2:
print("Success!")
```

```
Success!
Success!
```

Congratulations!

Congratulations on completing the assignment! Your dedication and effort are commendable. By successfully working through the coding exercises and written exercises, you have demonstrated a strong understanding of the concepts and principles related Information Retrieval.

Congratulations on finishing this notebook!

Acknowledgement

About the Author

This notebook was authored by Mohamed Reda Bouadjenek. He is a Senior Lecturer (Assistant Professor) of Applied Artificial Intelligence in the School of Information Technology at Deakin University, Australia.

Contact Information

- **Email:** reda.bouadjenek@deakin.edu.au
- **GitHub:** https://github.com/rbouadjenek/